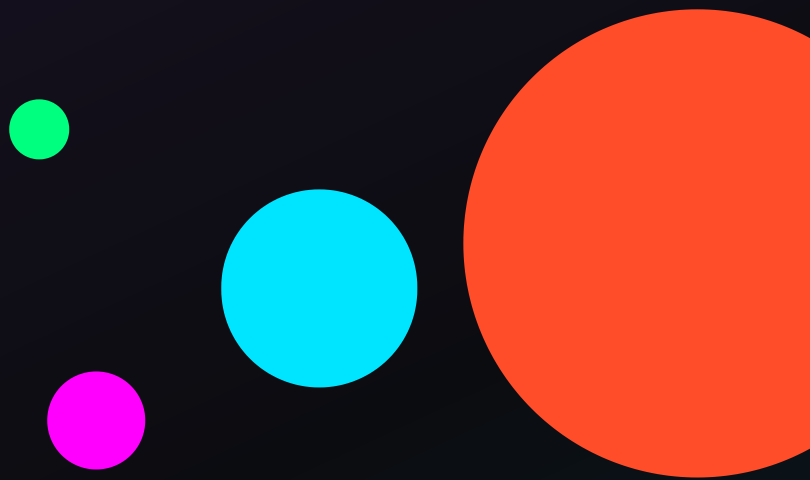
UYGULAMALI CSS & JAVASCRIPT

Bugün Ne Yesem? Kod Rehberi

Sitenin kodları neyi, nasıl ve neden çalıştırıyor? Zor kısımlar, gerçek satır numaraları ve denemelerle.



1 Sitenin genel işleyişi

2 JavaScript'in zor kısımları

3 CSS'in zor kısımları

4 Koddaki zayıf noktalar

5 Uygulamalı denemeler

BÖLÜM 1

Sitenin genel işleyişi

Dosyaların yüklenme sırası

`kiler.html:85`

```
<script src="data.js"></script>    // 1. recipes verisi
<script src="common.js"></script>  // 2. ortak fonksiyonlar
<script src="kiler.js"></script>   // 3. sayfanın kendi kodu
```

Sıra önemlidir, çünkü `kiler.js` önceki dosyalardaki `recipes` ve `norm`'u kullanır. Sıra değişirse sayfa `ReferenceError` verir.

Her sayfada aynı döngü

Veri değişir → **render() çalışır** → **HTML yeniden yazılır**

Ekranı tek tek düzeltmek yerine her seferinde veriden yeniden çizmek, React'in de temel fikridir.

BÖLÜM 2

JavaScript'in zor kısımları

Eşleştirme algoritması

`common.js:54`

Sitenin beyni burasıdır:

```
const pantry = getPantry().map(norm).filter(p => p.length >= 3);
const found = pantry.some(p => n.includes(p) || p.includes(n));
```

- **norm:** `toLocaleLowerCase("tr")` kullanır, yoksa "I" harfi "ı" yerine "i" olur ve Türkçe eşleşmeler bozulur.
- **İki yönlü includes:** Kilerdeki "Yağ", tarifteki "Sıvı Yağ" ile eşleşir.
- **length >= 3 filtresi:** Olmasa "su" yazınca "Kuru Fa**su**lye" de eşleşirdi.

İki kademeli sıralama

common.js:74

```
.sort((a, b) => a.missingIngredients.length - b.missingIngredients.length ||
                a.ingredients.length - b.ingredients.length)
```

- `sort` fonksiyonundan negatif sayı dönerse `a` öne, pozitif dönerse `b` öne geçer.
- Eksik sayıları eşitse fark `0` olur. `0` "yanlış" sayıldığı için `||` ikinci kritere geçer.
- Yön değiştirmenin kısa yolu `bilgi.js:37`'de: sonucu `sortDir` (1 veya -1) ile çarpmak sıralamayı ters çevirir.
- **Dikkat:** `sort` orijinal diziyi değiştirir. Bu yüzden `index.js:2` önce `[...recipes]` ile kopya alır; `bilgi.js:36`'da ise `filter` zaten yeni dizi döndürür.

Olay delegasyonu

common.js:140

```
document.addEventListener("click", e => {
  const btn = e.target.closest("[data-add-missing]");
  if (!btn) return;
```

- **Sorun:** `innerHTML` eski butonları silip yenilerini yaratır, butonlara tek tek bağlanan dinleyiciler de onlarla gider.
- **Çözüm:** Tıklama olayı yukarı doğru kabarcıklanır (bubbling), `document`'a kadar çıkar. Tek dinleyici en üstte durur.
- **closest:** Tıklanan yerden yukarı doğru arar. Butonun içindeki bir ikona tıklansa bile butonu bulur.

Aynı desen `kiler.js:59` ve `plan.js:53`'te de var.

data-* öznitelikleri

HTML'e veri gizlemenin yoludur. `data-add-missing="5"`, JS'te `btn.dataset.addMissing` olur (tireli ad camelCase'e döner). Değer her zaman metindir, bu yüzden kodda `Number(...)` var; `"5" === 5` yanlıştır.

Closure

tarif.js:88

```
function setupPortions(missingSet) {
  let portion = recipe.basePortion;
  function update() { /* portion'ı kullanır */ }
```

`setupPortions` bittikten sonra bile buton dinleyicileri `portion`'a erişebilir, çünkü iç fonksiyonlar doğdukları ortamı hatırlar. Böylece global değişken gerekmez. `tarif.js:23`'teki `draw` da aynı şekilde çalışır.

Hoisting

tarif.js:7

`renderList()` 7. satırda çağrılıyor ama 14. satırda tanımlanıyor. `function` bildirimleri dosyanın başına "kaldırılır", o yüzden çalışır. `const` ile tanımlananlarda bu olmaz.

LocalStorage

common.js:8

```
try {
  const v = JSON.parse(localStorage.getItem(key));
  return Array.isArray(v) ? v : [];
} catch (e) { return []; }
```

LocalStorage yalnızca metin saklar, bu yüzden `JSON.stringify` ve `JSON.parse` gerekir. `try/catch` ve `Array.isArray`, veri bozuksa sitenin çökmesini engeller.

- **const dizisi nasıl değişiyor?** `common.js:23`'teki `myPantry` `const` ama `kiler.js:53`'te `myPantry.length = 0` ile boşaltılıyor. `const` yalnızca değişkenin başka bir şeye atanmasını engeller, dizinin içini değil.
- **IIFE:** `common.js:23`'teki `(() => {...})()` kalıbı, tanımlanır tanımlanmaz çalışan fonksiyondur.

Sayfalar arası iletişim

index.js:45

```
location.href = `kiler.html?q=${encodeURIComponent(chip.dataset.q)}`;
```

Hedef sayfa bunu `kiler.js:8`'de `URLSearchParams` ile okur. `encodeURIComponent` Türkçe karakterleri URL'ye uygun hale getirir ("Öğle" → `%C3%96%C4%9Fle`).

Slayt döngüsü

index.js:19

```
current = (index + featured.length) % featured.length;
```

5 slaytta 0'dan geri gidince `-1` olur: `(-1 + 5) % 5 = 4`, yani son slayta döner. İleride `5 % 5 = 0` ile başa döner. `setInterval` / `clearInterval` ise fare üzerindeyken otomatik geçişi durdurur.

Bekleyen zamanlayıcıyı iptal etme

common.js:109

```
clearTimeout(toast.timer);
toast.timer = setTimeout(...);
```

Üst üste bildirim gelirse eski zamanlayıcı iptal edilir, yoksa yeni mesaj erken kapanırdı. Fonksiyonlar da nesne olduğu için `toast.timer` diye özellik eklenebilir.

Temanın yanıp sönmemesi

`theme.js:7`

Script, `<body>`'nin hemen başında yüklenir ve sınıfı sayfa çizilmeden ekler. Sonda yüklenseydi sayfa önce koyu görünüp sonra açılırdı.

BÖLÜM 3

CSS'in zor kısımları

CSS değişkenleriyle tema

`style.css:2, :36`

Renkler bir kez `:root`'ta tanımlanır, `body.light-mode` aynı değişkenleri ezer. Tema değiştirmek için tek bir sınıf yeterlidir, binlerce satır değişmez.

▶ Dene: document.body.classList.toggle("light-mode")

Split-screen

`style.css:96, :159`

```
.sidebar { position: fixed; width: var(--side-w); }
.split   { margin-left: var(--side-w); }
```

`fixed` eleman sayfa akışından çıkar, bu yüzden sağ taraf aynı genişlikte `margin` ile içeri itilir. `clamp(300px, 32vw, 480px)` ise "ekranın %32'si, ama 300 ile 480 piksel arasında" demektir.

Cam efekti ve yan etkisi

`style.css:249`

`backdrop-filter: blur()` arkadaki görüntüyü bulanıklaştırır, yarı saydam zemin de camı oluşturur. Ama iki yan etkisi var:

- **Yeni katman grubu:** Eleman yeni bir "stacking context" açar. Bu yüzden `style.css:257`'de `.pantry-add`'e `z-index: 5` gerekti, yoksa otomatik tamamlama listesi alttaki kartların arkasında kalırdı.
- **fixed çocuklar:** İçindeki `position: fixed` elemanlar artık ekrana göre değil, bu elemana göre konumlanır. Profil kutucuğu bu yüzden sidebar'ın dışında.

Özgüllük (specificity)

`style.css:560`

Filtre rengi hatasının sebebi buydu. Tarayıcı, seçicilerden daha özgül olanı seçer:

- `.btn` → 1 sınıf
- `.tags .btn-sm:not(.btn-outline)` → 3 sınıf, bu kazanır

Eşitlikte dosyada sonra gelen kazanır. `!important` (`style.css:322`) hepsini ezer ama sonradan ezilmesi çok zor olur, az kullanılmalı.

JS'siz davranış	Satır	Ne yapar
`input:checked + span`	style.css:635	Kutu işaretlenince yanındaki yazıyı değiştirir
`:empty { display: none }`	style.css:725	Boş öneri listesini gizler
`:not(:disabled)`	style.css:352	Pasif butonda hover'ı kapatır
`:has(body.light-mode)`	style.css:76	Üst elemanı, içindekine göre seçer

Özel onay kutusu

`style.css:617`

`appearance: none` tarayıcının kutusunu siler, `::after` ile ✓ çizilir. `transform: scale(0)` işareti gizler, `:checked` olunca `scale(1)` ve `transition` ile büyüyerek çıkar.

Otomatik grid

`style.css:522`

`repeat(auto-fill, minmax(280px, 1fr))` ile sütun sayısı ekrana göre ayarlanır, media query gerekmez.

Kenardan kenara görsel

`style.css:527`

Kartın `1.5rem` padding'i var. `margin: -1.5rem` ve `width: calc(100% + 3rem)` görseli kenarlara taşır.

Butonu dibe itme

`style.css:545`

Flex içinde `margin-top: auto`, kalan boşluğu yutar. Kartlar farklı boyda olsa da butonlar hizalı kalır.

Tam ortalama

`style.css:733`

`left: 50%` sol kenarı ortaya getirir, `translate(-50%)` elemanı kendi genişliğinin yarısı kadar geri çeker.

hidden tuzağı

`style.css:844`

`display: flex`, HTML'deki `hidden`'ı ezer. Bu yüzden `[hidden] { display: none }` ayrıca yazıldı.

Tıklamayı geçirme

`style.css:386`

`pointer-events: none`, süs ikonlarının altındaki butonlara tıklanabilmesini sağlar.

BÖLÜM 4

Koddaki zayıf noktalar

Güvenlik açığı: XSS

`kiler.js:21`

Kullanıcının yazdığı malzeme doğrudan `innerHTML`'e giriyor. Kilere `<img src=x onerror=alert(1)>` eklersen kod çalışır ve kayıtlı kaldığı için her açılışta tekrarlar. Doğrusu `kiler.js:95`'teki gibi `textContent` kullanmaktır. Veri paylaşılmadığı için risk düşük, ama bu hata türünü öğrenmeye değer.

Hatalı eşleşme

`common.js:60`

Kilerde "Domates Salçası" varsa iki yönlü `includes` yüzünden "Domates" de var sayılır. "Un" gibi kısa malzemeler ise 3 harf kuralına takılıp hiç eşleşmez.

keyup

`bilgi.js:43`

Fareyle yapıştırınca arama çalışmaz, `input` olayı daha doğrudur. Kiler sayfasında düzeltildi, burada istenen şekilde `keyup` bırakıldı.

Etkisiz kod

- `accent-color` (`style.css:823`), `appearance: none` yüzünden işe yaramıyor.
- `groups` dönüşümü (`plan.js:19`), veri zaten yemeğe göre gruplu olduğu için yalnızca alan adlarını değiştiriyor.

Ölü kod

`th.sortable` ve `.table-wrap` (`style.css:601`) ile `data.js`'teki `foods` dizisi artık kullanılmıyor.

BÖLÜM 5

Uygulamalı denemeler

Console'da (F12), kiler.html açıkken

```
findMatchingRecipes()[0]   // en az eksikli tarif
myPantry                   // kilerin içeriği
toast("Merhaba!")          // bildirimi elle çağır
localStorage               // kayıtlı veriler
```

CSS tarafında

Elements sekmesinde `.card`'ın `backdrop-filter` satırının tikini kaldır, cam efektinin kaybolduğunu gör. Sonra `:root`'ta `--accent` rengini değiştir, tüm sitenin aynı anda değiştiğini izle.

Kod tarafında, en kolaydan zora

- **1.** `common.js:55`'teki `>= 3`'ü `>= 1` yapıp kilere "su" ekle, hatalı eşleşmeyi gör.
- **2.** Bölüm 4'teki XSS açığını kapat.
- **3.** Besin sayfasına isme göre sıralama ekle.